Running the journal

The process, how it starts, its settings, its timer, and its tests.

Written from `journal/server.js`, `deploy-tools.sh` (the journal block), `tools.config.json` (the `JOURNAL` entry) and `tests/journal.*.test.js`.

The process

Name under pm2	`journal`
Command	`node journal/server.js`, from the repo checkout
Port	**3100**, from the `JOURNAL` entry in `tools.config.json` (`PORT` overrides it)
Memory limit	pm2 restarts it above `JOURNAL_MAX_MEM`, default **160 MB** (it uses about 8 MB)
Logs	pm2's, with timestamps (`--time`)
Started by	`./deploy-tools.sh`, which deletes and restarts it on every full deploy

The one-time move. On a deploy where `data/journal/journal.db` does not exist and the old separate install does (`$OLD_JOURNAL_DIR`, default `~/journal-app`), the deploy stops the old process, then **copies** its `tradedesk.db` (with any `-wal` / `-shm` files) to `data/journal/journal.db` and its `journal-accounts.json` (mode 0600). The old install is left untouched as a backup, and a second deploy changes nothing.

Start-up, in order

1. Open the database (`journal/src/db`), creating the old app's tables and settings (DATA-MODEL.md).
2. Create the setup and old trading tables (`trading/db.js`), then the grading tables (`trading/grading.js`).
3. Create the journal's own tables (`journal/db.js`), then add the always-blank `id` column to `journal_technical_snapshots` if it is missing. The repair must run after that table exists. Its first version ran too early, found no table, and silently did nothing.
4. Mount the login, the static files, the journal routes and the setup list. Loading the routes loads the check library, which seeds and repairs its 77 checks.
5. Mount the Alpaca accounts. A failure here is logged (`Alpaca accounts NOT mounted`) and the rest of the journal still runs.
6. Mount the app bar's three routes and `/`.

7. Listen. Requiring `server.js` from a test does not open a port.

The timer

One timer, from the accounts add-on: **every 60 seconds**, fetch each enabled Alpaca account whose interval has passed (ACCOUNTS.md). Nothing else in the journal runs on a clock. The technical compute runs only after a file import or when its routes are called.

Environment variables

Variable	Default	Used for
`PORT`	3100	the port
`JOURNAL_DATA_DIR`	`<repo>/data/journal`	the database and the accounts file
`JOURNAL_DB`	`<JOURNAL_DATA_DIR>/journal.db`	the database alone
`SHARED_KEYS_FILE`	`<repo>/data/keys.json`	the fallback Alpaca key for bars
`JOURNAL_MAX_MEM`	`160M`	pm2's restart limit (deploy only)
`OLD_JOURNAL_DIR`	`~/journal-app`	where the one-time move copies from (deploy only)

Outside calls

To	When	Key
Alpaca trading API (`paper-api` or `api`)	each account fetch and key test	the account's own
Yahoo chart API	the technical compute	none
Alpaca market data	the technical compute, when Yahoo fails	the fallback chain (ANALYSIS.md)

The journal never calls the screener tools or qp. The only link between them is the 📈 button, a plain link to qp's print sheet in a new tab.

Tests

`npx jest tests/journal` runs **7 files, 84 tests**. All pass on the code as documented.

File	Tests	Covers
`journal.alpacaAccounts.test.js`	18	the accounts add-on on a real express app and SQLite, against a fake Alpaca: no secret leaks, stable ids, closed trades never rewritten, the schedule, refused keys
`journal.import.test.js`	19	**the Alpaca fill pairing** (`fillsToTrades.js`): weighted prices, reversals, open trades, ids
`journal.accounts.test.js`	17	`addons.js` run against a DOM stub: the account and setup filters, remembered choices that no longer exist
`journal.appbar.test.js`	13	the real server's `/deskbar.css` slice, `/desk.js` , `/api/tools`
`journal.calendar.test.js`	11	the calendar grid: right weekday column, one week cell per week
`journal.closedQueries.test.js`	4	the metrics, calendar and compute-all queries run (they once failed on every call)
`journal.robust.test.js`	2	the grading backfill survives an untagged trade; bars can be fetched without a `trading_brokers` table

Not covered by any test: the file importer (`importer.js`), where five of the six cases tried while writing these documents go wrong (IMPORT.md); the technical compute; the condition rules; the page's metric formulas. A rebuild should start its tests there.